

SD TCL - The ed Line Editor

The editor that runs inside SD, needs no terminal, and can be driven from a script.

SD has three editors and they are all installed with it. **ed** is the one that runs *inside SD*: it is written in SD BASIC, draws nothing, and needs neither a terminal nor operating-system permission. That is what makes it the editor for a phantom, an API session, a session driven down a pipe, and anything you want to automate.

The other two are edit and micro, which hand the record to a full-screen Windows program. Use one of those when you are sitting at a terminal and want to see the whole record at once.

SD folds case, so a command may be typed in either case. Commands are shown here in lower case. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part. **ed's own commands are shown in upper case**, which is how **ed**'s **HELP** prints them; it folds case too.

Every listing on this page was produced by running it, on SD Core for Windows W1.0-0, against a scratch record in an account's **bp** file.

Starting it

```
ed {dict} file {record...} {no.query}
```

Naming the record opens it straight away:

```
ed bp zzed
```

```
bp zzed  
New record  
----:
```

----: is the prompt. A record that exists reports its length instead:

```
bp zzed  
3 line(s)
```

Omit the record name and `ed` **asks for one**, and asks again after each edit until you give it an empty answer. **Give several names and it works through them in order** — the record after the current one is reached by filing or quitting the one you are on:

```
ed bp zzed zzedb
```

```
bp zzed
3 line(s)
----: Q
bp zzedb
New record
----:
```

A file name is tried as typed, then lower case, then upper. A file that matches none of the three stops the verb before the editor starts:

```
ed nosuchfile zzed
```

```
File not found
```

`dict` edits the file's dictionary rather than its data. It is the same addressing as everywhere else, and a dictionary with no such entry is a new record like any other:

```
ed dict voc who.am.i
```

```
DICT voc who.am.i
New record
```

The commands

MAKING A RECORD FROM NOTHING

`I` inserts one line after the current one. At the top — where a new record starts — it inserts at line 1:

```
ed bp zzed
I * ZZED - a scratch record for the SD TCL editing page
```

```
I crt 'ZZED OK'
I end
T
P20
FI
```

```
bp zzed
New record
----: I * ZZED - a scratch record for the SD TCL editing page
Bottom at line 1
----: I crt 'ZZED OK'
Bottom at line 2
----: I end
Bottom at line 3
----: T
Top
----: P20
0001: * ZZED - a scratch record for the SD TCL editing page
0002: crt 'ZZED OK'
0003: end
Bottom at line 3
----: FI
'zzed' filed in bp
```

I **with no text** starts a multi-line insert instead, prompting `0001=` for each line until you enter an empty one. **IB** is the same as **I** but inserts before the current line rather than after it.

MOVING ABOUT

T	to the top — <i>before</i> line 1, so I inserts at 1
----------	---

B	to the last line
----------	------------------

n	to line <i>n</i>
----------	------------------

+ n , - n	<i>n</i> lines forward or back
-------------------------	--------------------------------

G n , P0 n	to line <i>n</i> , the long ways round
--------------------------	--

G< , G>	to the first or last line of the block
-----------------------------	--

LOOKING

P <i>n</i> *	display <i>n</i> lines from the current line
PL { - } <i>n</i>	<i>n</i> lines relative to the current line
PP <i>n</i> *	<i>n</i> lines centred on the current line
PB	the block
L <i>string</i>	the next line containing <i>string</i>
F { <i>col</i> } <i>string</i>	the next line with <i>string</i> at column <i>col</i> , default 1
M <i>pattern</i>	the next line matching a pattern
COL	a column ruler
?	where you are and what is set

P leaves the current line at the last line it printed, and that is what the next command acts on. It is the one thing on this page that catches people, because "display" does not sound like a move. **P20** on a three-line record, then **DE**, and it is **line 3** that goes:

```
----: P20
0001: * ZZED - a temporary record for the SD TCL editing page
0002: crt 'ZZED OK'
0003: end
Bottom at line 3
----: DE
Bottom at line 2
```

? is worth knowing before you ask anyone else:

```
----: ?
File name           = bp
Record name         = zzed
Current line        = 3
No block defined
OOPS will undo 'I end'
Searches are case-sensitive
Expansion of non-printing characters : Disabled
Non-printing character entry          : Disabled
Verification of block actions         : Enabled
```

Searches are case-sensitive until you say otherwise — `CASE OFF` makes `L` and `F` insensitive, `CASE ON` puts it back.

CHANGING

<code>I {text}</code> , <code>IB {text}</code>	insert after, insert before
<code>R text</code>	replace the current line
<code>A {text}</code>	append to the end of the current line
<code>C/ old / new / {n}{G}{B **}</code>	change <i>old</i> to <i>new</i> in <i>n</i> lines. <code>G</code> every occurrence, <code>B</code> over the block
<code>R/ old / new** / ...</code>	the same, without moving
<code>D n</code> , <code>DE n</code>	delete <i>n</i> lines from the current one
<code>DUP {n}</code>	duplicate the current line
<code>CAT {string}</code>	join the next line onto this one
<code>FORMAT</code>	re-indent BASIC source

The delimiter of a `C` command is whatever character follows the `C`, so a change involving `/` can use `,` or `:` instead.

`OOPS` UNDOES THE LAST THING THAT CHANGED THE RECORD

One step, and it names what it is undoing:

```
----: OOPS
Undoing 'DE'
```

`?` tells you in advance what it would undo, which is the line above reading `OOPS will undo 'I end'`.

BLOCKS

`<` sets the start of a block at the current line, `>` the end, `<>` makes the current line a one-line block. Then `COPY` copies it after the current position, `MOVE` copies and deletes the original, `DROP` deletes it, and `PB` displays it. `<` at the top clears the block.

VALUES ON A LINE

A field is a line, so `ed` handles fields on its own. **A value mark and a subvalue mark are single unprintable characters**, and a line holding them displays as one run with a stray glyph in it. `EV` opens the current line as if its values were lines:

```
----: EV
Editing values. Use SV to save changes or QV to discard changes
----: P20
0001: NAME
0002: Smith
0003: Jones
Bottom at line 3
----: QV
EV mode changes discarded
Returned from EV mode
```

SV leaves **EV** keeping the changes, **QV** discards them. Three levels are available: values, subvalues, and the elements of a compound l-type.

A mark is typed as ^ and its character code: ^253 for a value mark, ^252 for a subvalue mark, ^^ for a literal ^. The line above was made with `I NAME^253Smith^253Jones`.

LEAVING

FI	write the record and end the edit
FIB	file it, then basic it
FIBR	file it, compile it, and run it
SAVE	write it and stay in the editor
Q , QUIT	end the edit, discarding changes
X	abandon the whole run when a select list is in use
FD , DELETE	delete the record from the file and end the edit

```
----: FI
'zzed' filed in bp
```

Q asks only **If you changed something**, and it asks on the same line:

```
----: Q
Record changed - OK to quit? Y
```

Anything but **Y** and you are still in the editor. On an unchanged record **Q** says nothing at all and returns to TCL.

SAVE and **FI** with a different file or record name ask before they overwrite, and so do **FD** and **DELETE**.

Written back to their own record they ask nothing at all, which is what lets **ed** be driven from a script.

XEQ RUNS A TCL COMMAND WITHOUT LEAVING THE EDITOR

@FILE, **@ID** and **@LINE** are substituted, so a command can be about whatever you are editing:

```
----: XEQ ct @FILE @ID
bp zzed
1: * ZZED - a scratch record for the SD TCL editing page
2: crt 'ZZED OK'
3: end
```

And what it showed there was the filed record, not the one on the screen. The buffer at that moment read "a temporary record" — the change had not been filed. **XEQ** runs outside the editor and sees the file, which is the answer you want from **basic** and the wrong one from **ct**.

HELP

HELP alone prints every command. **HELP** with a topic prints one:

```
----: HELP FI
File [[filename] record.id]
      Write record and end edit
```

THE COMMAND STACK, AND PRESTORED SEQUENCES

Every command you type is kept. **.L** lists the stack, **.R** *n* recalls line *n* to the top, **.X** *n* executes it again, and **.A**, **.C**, **.I** and **.D** append to, change, insert and delete stack lines.

.X with a file and record instead of a number runs a *prestored edit sequence* — a record holding one **ed** command per line:

```
ed bp zzseq
I ED
I L scratch
I C/scratch/altered/
I FI
FI
```

```
ed bp zzed
```

```
.X bp zzseq
```

```
bp zzed
3 line(s)
----: .X bp zzseq
0001: * ZZED - a scratch record
0001: * ZZED - a altered record
'zzed' filed in bp
```

Field 1 is a marker, not a command. it must begin with `E` — `ED` is the usual thing to write — and `ed` refuses a record that does not with "*Record is not a prestored edit sequence*". Nothing was run for it above, and the line numbers in error messages count it: an unrecognised command in **field 2** is reported as line 2.

`PAUSE` inside a sequence stops it and `.XR` resumes, `.XK` abandons it; `LOOP` repeats from a given line.

A prestored sequence is not an unattended mechanism. One bad command **abandons the rest of the sequence and then asks a question**, and the answer is read from wherever the session's input comes from.

```
----: .X bp zzs2 Error - Unrecognised command at line 2 of prestored commands Did you mean to insert
this text (Y/N)? N
```

The same is true of a multi-line `I`, which is refused outright inside a sequence. **If you are driving `ed` down a pipe, give every command its arguments** — `I` with text, `FI` with no name, never `FD` — and it runs through without stopping.

Who has it

`ed` **is a programmer verb**. A standard account does not have it, and does not have `edit` or `micro` either — an account that runs an application does not edit the records behind it.

See also

SD TCL - The edit Screen Editor · SD TCL - The micro Screen Editor · SD TCL - Programs and the Catalogue.